

Sl. no	INDEX	Page no.
1	INTRODUCTION TO ROBOTICS	1
2	APPLICATION OF ROBOTICS AND NEED OF ROBOTICS	2
3	OBJECTIVES	3
4	TYPES OF ROBOTICS SYSTEM	4
5	HARDWARE AND SYSTEM ARCHITECTURE OF ROBOTICS	5
6	METHODOLOGY AND CONTROL	6
7	HUMAN-ROBOT INTERACTION	7
8	INTRODUCING THE ESP32 AND ARDUINO UNO	9
9	PROJECT TECHNICAL DESIGN & DOCUMENTATION	11
	TECHNICAL SPECIFICATIONS	11
	REMOTE CONTROLLER UNIT (TRANSMITTER) 2.1 Components Used 2.2 Connections	12
	RECEIVER VEHICLE UNIT (CAR) 3.1 Components Used 3.2 Receiver Vehicle Hardware Interfacing Map	15
	FIRMWARE OPERATION 4.1 Transmitter Operation 4.2 ESP 32 Code 4.3 Receiver Operation 4.4 Arduino Code 4.5 System functional table	26
10	FUTURE APPLICATIONS	40
11	CONCLUSION	42

INTRODUCTION TO ROBOTICS

Robotics is the interdisciplinary study and practice of the design, construction, operation, and use of robots. A robot is a machine, especially one programmable via a computer, capable of automatically carrying out a complex series of actions. A robot can be guided by an external or internal control device. Robots may be humanoid, but most are task-performing machines prioritizing functionality over aesthetics. A roboticist is someone who specializes in robotics. Robotics usually combines four aspects of design work: a power source (e.g. a battery), mechanical construction, a control system (electrical circuits), and software (run by remote control or artificial intelligence).

The goal of most robotics is to design machines that can assist human in various fields, such as agriculture, construction, domestic work, food processing, inventory management, manufacturing, medicine, military, mining, space exploration, and transportation.

APPLICATION OF ROBOTICS AND NEED OF ROBOTICS

The increasing complexity of industrial, medical, and scientific tasks has created a strong demand for robotic systems. Robotics is needed to improve productivity, accuracy, efficiency, and safety. Robots can work continuously without fatigue, perform repetitive tasks with high precision, and operate in environments that are dangerous for humans. In industries, robots reduce human errors and increase production rates. In hazardous areas such as nuclear plants, mines, and disaster zones, robots can perform tasks while protecting human lives. Robotics also addresses labor shortages and supports technological advancement in modern society.

Applications of Robotics

1. Industrial Automation

Robots are widely used in manufacturing industries for welding, painting, assembly, packaging, and quality inspection. They improve production speed and maintain consistent product quality.

2. Healthcare and Medicine

Medical robots assist surgeons in performing complex operations with high precision. Robots are also used for rehabilitation, patient monitoring, medicine delivery, and hospital automation.

3. Agriculture

Agricultural robots help in planting, harvesting, irrigation, crop monitoring, and pesticide spraying. These robots increase efficiency and reduce manual labor.

4. Space Exploration

Space agencies use robotic rovers and robotic arms to explore planets, collect samples, and perform maintenance tasks in space environments where human presence is difficult.

5. Defense and Security

Robots are used for surveillance, bomb disposal, border monitoring, and reconnaissance missions. They help reduce risks to military personnel.

6. Service and Domestic Applications

Household robots such as robotic vacuum cleaners, lawn mowers, and personal assistant robots make daily tasks easier and more convenient.

7. Education and Research

Educational robots help students learn programming, electronics, and automation concepts. Researchers use robots to study artificial intelligence and autonomous systems.

OBJECTIVES

1. Automation of Repetitive Tasks : To perform repetitive and routine tasks automatically with minimal human intervention.
2. Improving Accuracy and Precision : To achieve high precision and consistency in operations such as manufacturing and surgery.
3. Enhancing Safety : To perform dangerous tasks in hazardous environments and reduce risks to human life.
4. Increasing Productivity and Efficiency : To work continuously without fatigue, thereby improving productivity and reducing operational time.
5. Assisting Humans in Complex Tasks : To support humans in physically demanding, difficult, or highly specialized activities.
6. Exploration and Scientific Research : To explore inaccessible environments such as space, deep oceans, and disaster zones.
7. Advancing Artificial Intelligence and Innovation : To develop intelligent systems capable of learning, decision-making, and autonomous operation.

TYPES OF ROBOTICS SYSTEM

Introduction

Robotic systems are machines designed to perform specific tasks automatically or with minimal human intervention. Depending on their design and application, robotic systems are classified into different types.

Types of Robotic Systems

1. Industrial Robots : Used in factories for welding, painting, assembly, packaging, and material handling.
2. Service Robots : Assist humans in homes, hospitals, hotels, and offices by performing service-related tasks.
3. Mobile Robots : Robots capable of moving from one location to another using wheels, tracks, or legs.
4. Humanoid Robots : Robots designed to resemble and imitate human appearance and behavior.
5. Medical Robots : Used in healthcare for surgery, rehabilitation, patient monitoring, and medicine delivery.
6. Military and Defense Robots : Used for surveillance, bomb disposal, reconnaissance, and border security operations.
7. Autonomous Robots : Robots that can sense their environment, make decisions, and operate without continuous human control.

Conclusion

Different types of robotic systems are developed to meet specific needs in industries, healthcare, defense, research, and daily life. These robots improve efficiency, safety, and productivity while reducing human effort.

HARDWARE AND SYSTEM ARCHITECTURE OF ROBOTICS

Introduction

The hardware and system architecture of a robotic system define how different components work together to perform tasks. A robot consists of sensing, processing, control, and actuation units that interact to achieve desired operations.

Hardware Components of a Robotic System

1. **Sensors** : Used to detect environmental information such as distance, temperature, light, pressure, and motion.
2. **Controller** : The brain of the robot that processes sensor data and makes decisions. Examples: Arduino, ESP32, STM32, Raspberry Pi.
3. **Actuators** : Convert electrical signals into physical movement. Examples: DC motors, servo motors, and stepper motors.
4. **Power Supply** : Provides electrical energy to all robot components through batteries or power adapters.
5. **Communication Module** : Enables data exchange between robots and other devices. Examples: Bluetooth, Wi-Fi, ZigBee, and RF modules.
6. **Mechanical Structure** : The physical body or frame that supports all hardware components and provides stability.
7. **End Effector** : The tool attached to the robot for performing specific tasks such as gripping, welding, or cutting.

System Architecture of a Robotic System

1. **Sensing Layer** : Collects data from the environment using sensors.
2. **Processing Layer** : Analyzes sensor data and generates decisions using the controller.
3. **Control Layer** : Converts decisions into control signals for actuators.
4. **Actuation Layer** : Executes the required movement or action using motors and actuators.
5. **Communication Layer** : Handles information exchange between the robot, user, and external systems.

Conclusion

The hardware and system architecture of robotics consist of sensors, controllers, actuators, communication modules, and power systems working together. This architecture enables robots to sense, process, decide, and act efficiently in various applications.

METHODOLOGY AND CONTROL

Introduction

Methodology and control are essential parts of a robotic system. Methodology defines the sequence of steps a robot follows to perform a task, while the control system ensures accurate and efficient operation.

Methodology of a Robotic System-

1. Sensing : The robot collects information from its environment using sensors.
2. Data Processing : The controller processes the sensor data and analyzes the situation.
3. Decision Making : Based on the processed data, the robot determines the appropriate action.
4. Control Signal Generation : Control commands are generated for the actuators.
5. Action Execution : Motors and actuators perform the required movement or task.
6. Feedback Monitoring : The robot continuously monitors the results using sensors and adjusts its actions if necessary.
7. Task Completion : The robot completes the assigned operation and waits for the next command or task.

Control of a Robotic System-

1. Open-Loop Control : The robot performs actions without receiving feedback from the environment.
2. Closed-Loop Control : The robot uses sensor feedback to continuously adjust and improve performance.
3. Manual Control : The robot is directly controlled by a human operator.
4. Autonomous Control : The robot makes decisions and operates independently without human intervention.
5. Intelligent Control : The robot uses Artificial Intelligence (AI) and Machine Learning (ML) for adaptive decision-making.

Conclusion

Methodology provides the workflow of a robotic system, while control systems ensure accurate and reliable operation. Together, they enable robots to perform tasks efficiently, safely, and intelligently.

HUMAN-ROBOT INTERACTION

Human-Robot Interaction (HRI) is the study of communication and collaboration between humans and robots. It focuses on designing robots that can interact with people safely, efficiently, and naturally. HRI enables robots to understand human commands through voice, gestures, touch, or visual signals and respond appropriately. It is widely used in healthcare, manufacturing, education, service industries, and assistive technologies. Effective Human-Robot Interaction improves user experience, enhances productivity, and allows humans and robots to work together efficiently in various applications.

Sensor Fusion :

Sensor Fusion is the process of combining data from multiple sensors to obtain more accurate, reliable, and complete information about the environment than could be achieved using a single sensor. It helps reduce errors, improve decision-making, and enhance the performance of robotic systems.

In robotics, sensor fusion combines information from sensors such as cameras, ultrasonic sensors, LiDAR, GPS, accelerometers, and gyroscopes. By integrating data from different sources, robots can better understand their surroundings, improve navigation, detect obstacles accurately, and perform tasks more efficiently. Sensor fusion is widely used in autonomous vehicles, drones, industrial robots, and mobile robots to increase accuracy, reliability, and operational safety.

Swarm Robotics :

Coordinating multiple robots to work as a single system.

Swarm Robotics is a branch of robotics in which multiple robots work together as a coordinated group to accomplish a common task. It is inspired by the collective behavior of social insects such as ants, bees, and termites. Each robot in the swarm operates independently but communicates and cooperates with other robots to achieve the desired objective.

Swarm robotics offers advantages such as scalability, flexibility, fault tolerance, and efficient task distribution. Even if one robot fails, the remaining robots can continue the operation. Swarm robotics is widely used in search and rescue missions, environmental monitoring, agricultural automation, military surveillance, warehouse management, and space exploration, where large-scale coordination and adaptability are required.

Ethics Of AI In Robotics :

Safety protocols and decision-making frameworks.

Ethics of AI in Robotics refers to the moral principles and guidelines that govern the design, development, and use of intelligent robotic systems. It ensures that robots operate safely, fairly, and responsibly while respecting human rights and societal values.

Key ethical concerns include safety, privacy, accountability, transparency, and bias in decision-making. AI-powered robots should not cause harm to humans, misuse personal data, or make unfair decisions. Developers must ensure that robotic systems are reliable, secure, and used for the benefit of society. Ethical AI in robotics promotes trust, responsible innovation, and the safe integration of robots into everyday life.

Implementation:

The implementation of robotics involves the integration of hardware and software components to perform specific tasks automatically. A typical robotic system consists of sensors, controllers, actuators, communication modules, and a power supply. Sensors collect information from the environment, which is processed by the controller to make decisions. Based on these decisions, actuators such as motors execute the required actions.

The implementation process includes system design, hardware integration, programming, testing, and deployment. Modern robots use technologies such as artificial intelligence, machine learning, computer vision, and sensor fusion to improve performance and adaptability. Robotics is implemented in various fields including manufacturing, healthcare, agriculture, transportation, defense, and service industries to increase efficiency, accuracy, and safety.

INTRODUCING THE ESP32 AND ARDUINO UNO

Exploring the Arduino Uno R3 :

The Arduino Uno R3 is a popular microcontroller board, often used as an entry point into electronics and programming. It features a simple design, an easy-to-use software environment, and a large community of users, making it ideal for prototyping and DIY projects. The board includes digital and analog input/output pins, a USB connection for programming and power, and the ATmega328P microcontroller, allowing users to control LEDs, read sensors, drive motors, and much more. It's an excellent tool for learning about circuits and embedded systems.



Introduction To Esp32 :

The ESP32 (Doit DevKit V1) is a high-performance, low-cost System-on-Chip (SoC) microcontroller designed by Espressif Systems. It features a powerful 32-bit dual-core Tensilica Xtensa LX6 processor operating at speeds up to 240 MHz. Unlike standard microcontrollers, the ESP32 is engineered with native, integrated 2.4 GHz Wi-Fi and Bluetooth (Classic and BLE) stacks, making it an industry-standard choice for wireless communication and Internet of Things (IoT) architectures.



In this project, the ESP32 serves as the Master Control Remote Node. It utilizes its processing power to manage two core real-time tasks simultaneously:

Contactless Edge Computing:

The ESP32 triggers and samples a dual HC-SR04 ultrasonic sensor array, calculating pulse time-of-flight measurements to capture hand gestures. It executes embedded linear interpolation algorithms (`map()`) to convert raw distance constraints into precise dynamic speed adjustments.

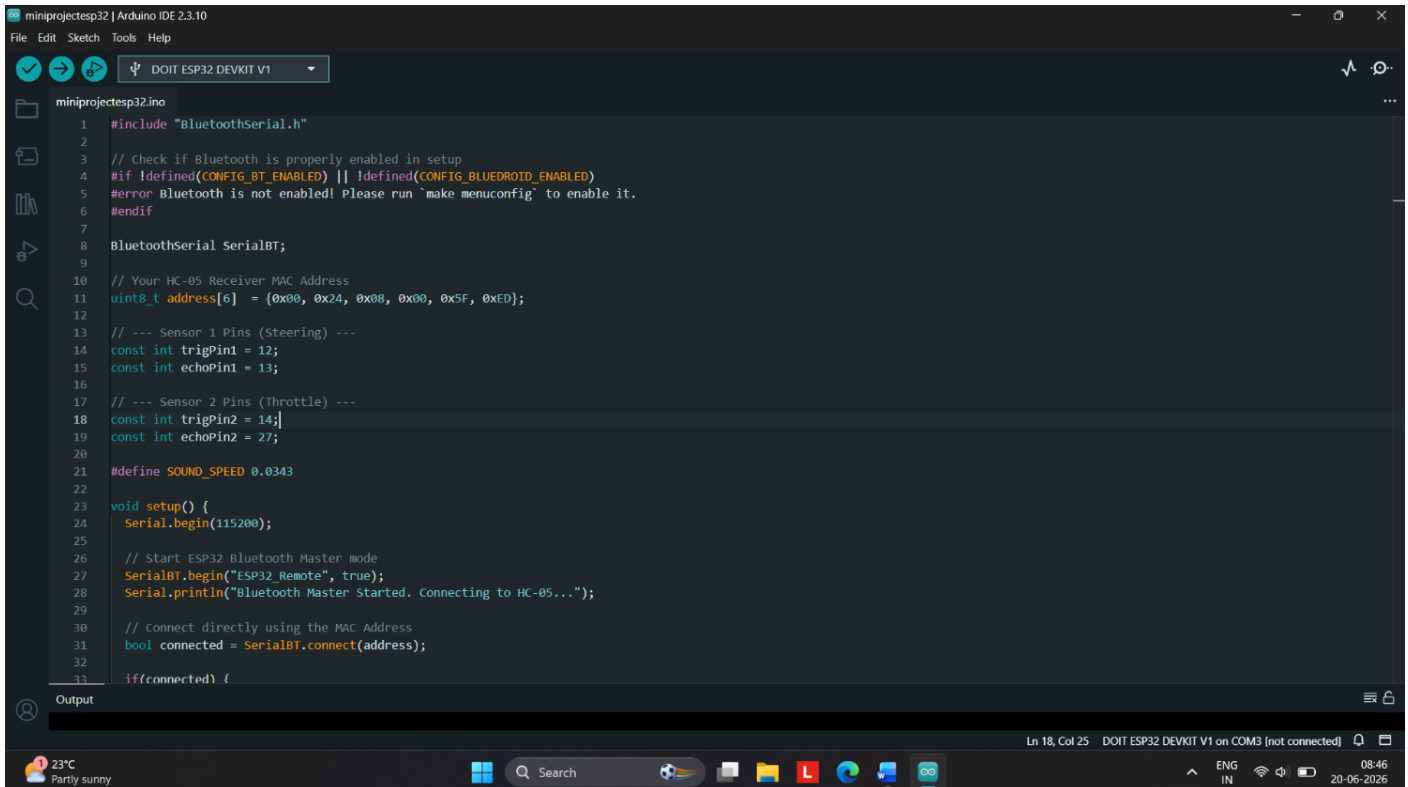
Bluetooth Master Broadcasting:

Utilizing its built-in Classic Bluetooth radio stack via the `BluetoothSerial` library, the ESP32 behaves as a dedicated wireless master node. It uses the specific hardcoded MAC address of the car's HC-05 module to automatically find, pair, and establish a secure, low-latency, over-the-air communication link without needing external wireless hardware. Once connected, it packages the gesture data into structured CSV packets and streams them continuously to the receiving vehicle.

PROJECT TECHNICAL DESIGN & DOCUMENTATION

1. Technical Specifications

Coding IDE Used: Arduino IDE 2.x configured with ESP32 Core by Espressif Systems and AVR Core for Arduino Uno.



```
miniprojectesp32 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
DOIT ESP32 DEVKIT V1

miniprojectesp32.ino
1 #include "BluetoothSerial.h"
2
3 // Check if Bluetooth is properly enabled in setup
4 #if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
5 #error Bluetooth is not enabled! Please run 'make menuconfig' to enable it.
6 #endif
7
8 BluetoothSerial SerialBT;
9
10 // Your HC-05 Receiver MAC Address
11 uint8_t address[6] = {0x00, 0x24, 0x08, 0x00, 0x5F, 0xED};
12
13 // --- Sensor 1 Pins (Steering) ---
14 const int trigPin1 = 12;
15 const int echoPin1 = 13;
16
17 // --- Sensor 2 Pins (Throttle) ---
18 const int trigPin2 = 14;
19 const int echoPin2 = 27;
20
21 #define SOUND_SPEED 0.0343
22
23 void setup() {
24   Serial.begin(115200);
25
26   // Start ESP32 Bluetooth Master mode
27   SerialBT.begin("ESP32_Remote", true);
28   Serial.println("Bluetooth Master Started. Connecting to HC-05...");
29
30   // Connect directly using the MAC Address
31   bool connected = SerialBT.connect(address);
32
33   if (connected) {
```

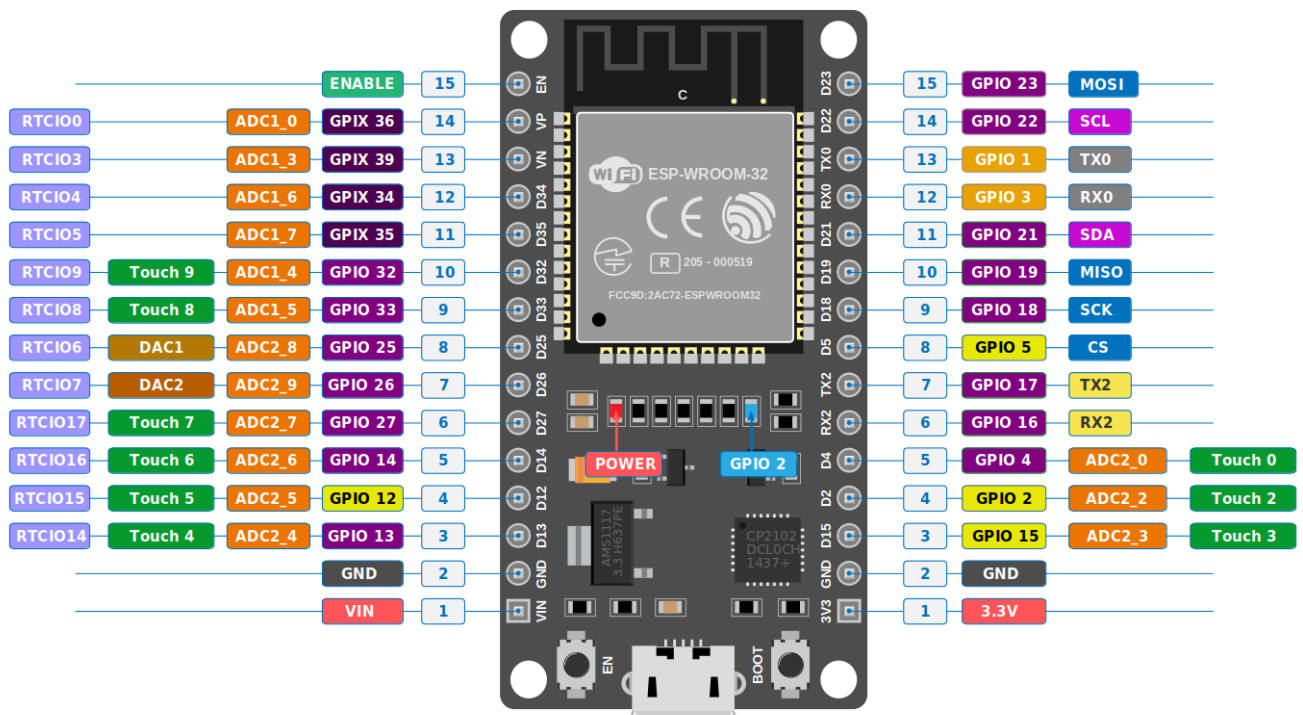
Programming Language Used : Embedded C++ (Object-Oriented and Procedural Hybrid Firmware Architecture)
Operating System used : Windows

2. Remote Controller Unit (Transmitter)

2.1 Components Used

ESP32 Doit DevKit V1 Microcontroller

A 32-bit dual-core microcontroller operating at 240 MHz. It processes sensor data in real time and provides built-in wireless Bluetooth communication.

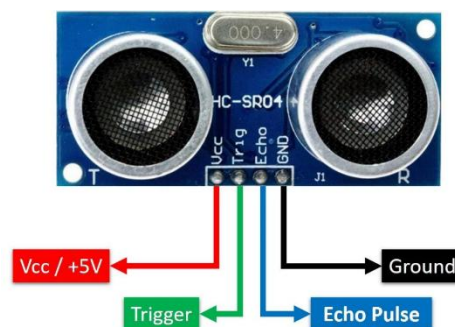


HC-SR04 Ultrasonic Sensor 1 (Steering Control)

Measures hand position using ultrasonic waves and controls vehicle steering based on distance.

HC-SR04 Ultrasonic Sensor 2 (Throttle Control)

Measures hand position and converts the detected distance into speed and direction commands.



- The VCC pin serves as the primary power source for the HC-SR04 ultrasonic sensor. It requires a stable 5-volt direct current (DC) supply to power the internal circuitry, transmitter, and receiver components. This pin is typically connected directly to the 5V power output rail of a microcontroller, such as an Arduino, or an external regulated power supply.
- The Trigger (Trig) pin acts as the input control line that commands the sensor to begin a distance measurement cycle. To initiate a reading, the host microcontroller must send a precise 10-microsecond high-voltage pulse to this pin. Upon receiving this specific signal, the sensor's internal clock triggers the transmitter to emit a burst of eight consecutive ultrasonic sound pulses at a frequency of 40 kilohertz.
- The Echo pin is the output signal line that communicates the travel time of the sound waves back to the microcontroller. This pin remains at a low voltage state until the ultrasonic burst is sent, at which point it immediately flips to a high voltage state. It stays high until the reflected sound waves bounce off an object and return to the receiver, meaning the duration of this high signal directly represents the total round-trip time of the sound.
- The Ground (GND) pin completes the electrical circuit and provides a common reference point for all signals. It must be connected directly to the ground rail of the microcontroller or power supply powering the sensor. Ensuring a solid, shared ground connection across all connected components prevents electrical noise and guarantees that the voltage pulses on the Trig and Echo lines are read accurately.

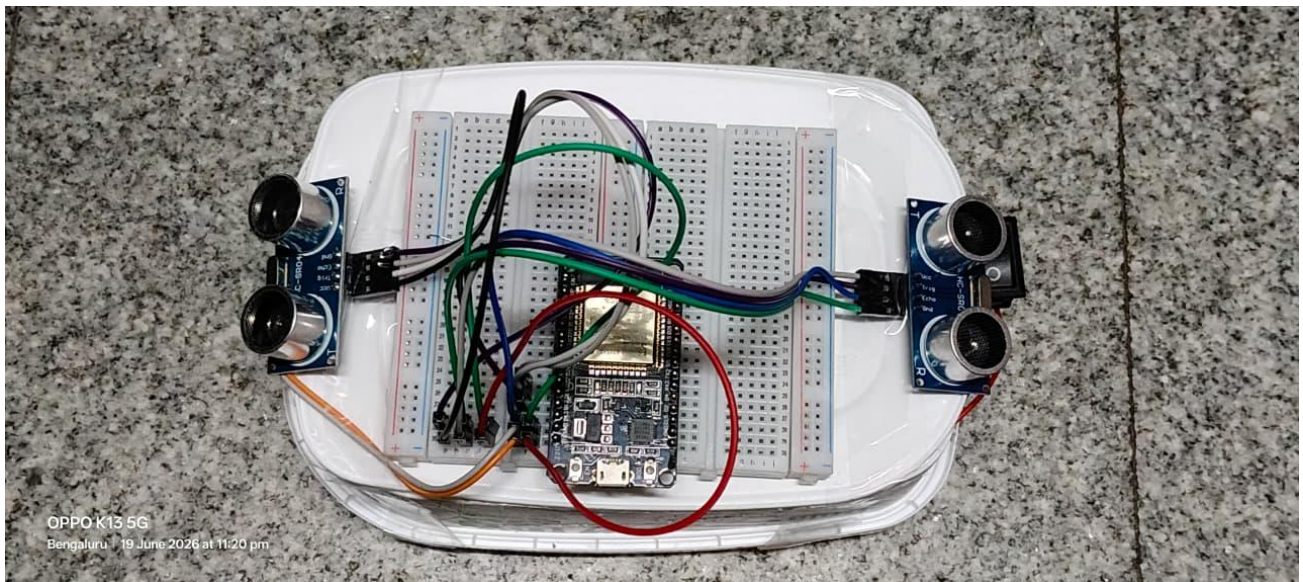
2.2 Connections

Sensor 1 (Steering)

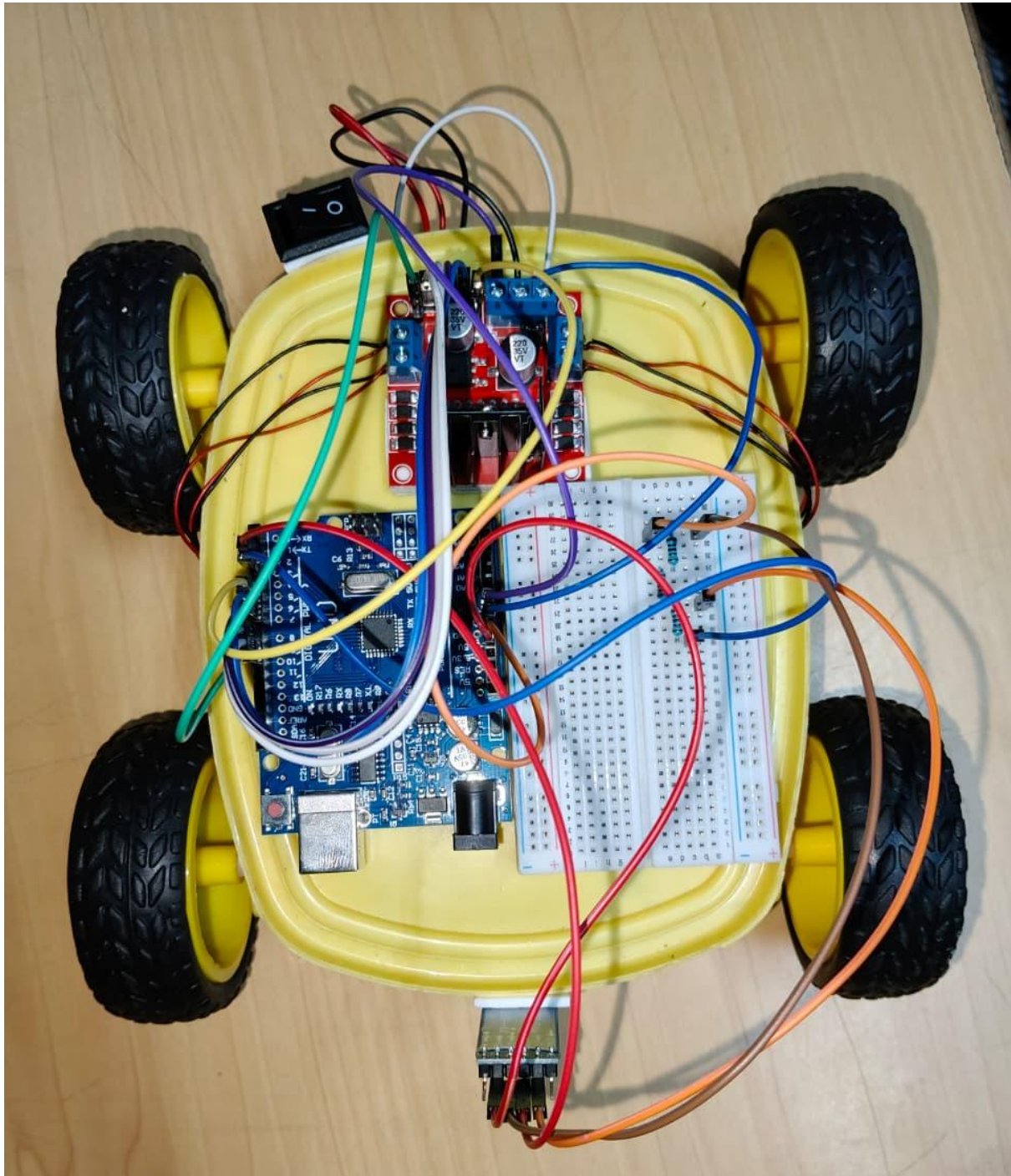
- VCC -> ESP32 VIN (5V)
- GND -> ESP32 GND
- Trig -> GPIO 12
- Echo -> GPIO 13

Sensor 2 (Throttle)

- VCC -> ESP32 VIN (5V)
- GND -> ESP32 GND
- Trig -> GPIO 14
- Echo -> GPIO 27



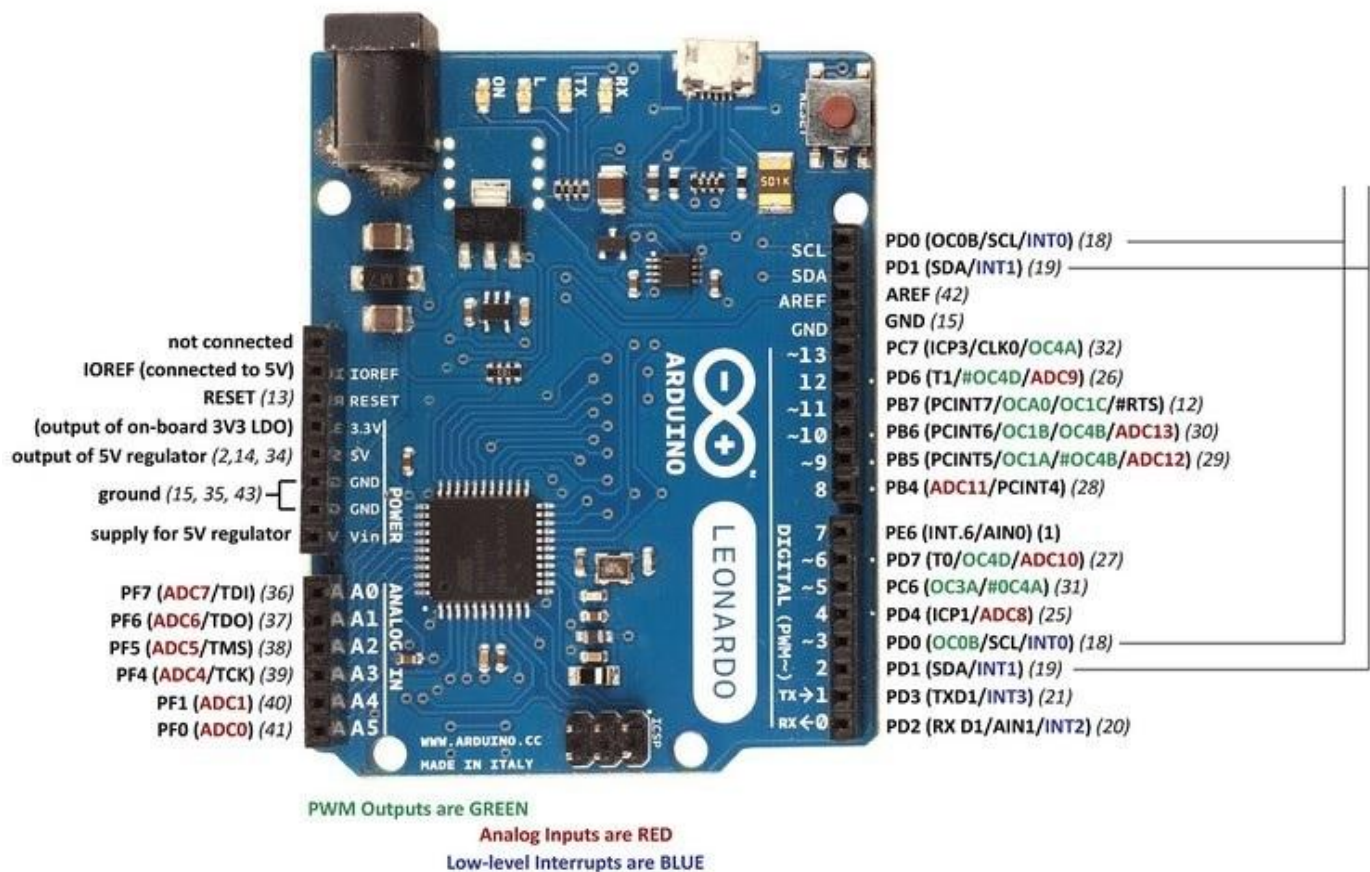
3. Receiver Vehicle Unit (Car)



3.1 Components Used

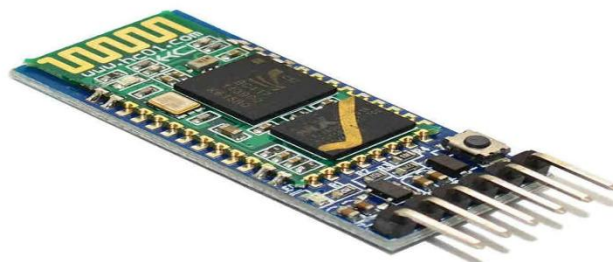
Arduino Uno Rev3 :

An 8-bit AVR microcontroller operating at 16 MHz. It receives commands from Bluetooth and controls the motors.



HC-05 Bluetooth Module :

Acts as a Bluetooth receiver and transfers wireless commands to the Arduino through serial communication.



HC-05 Bluetooth Module Quick Reference Guide

The HC-05 is a popular, inexpensive Bluetooth SPP (Serial Port Protocol) module designed for transparent wireless serial communication between microcontrollers (like Arduino, Raspberry Pi, ESP32) and Bluetooth-enabled devices (like smartphones and laptops).

1. Technical Specifications

Bluetooth Protocol: Bluetooth V2.0 + EDR (Enhanced Data Rate)

Frequency Band: 2.4 GHz ISM Band

Modulation: GFSK (Gaussian Frequency Shift Keying)

Emission Power: Class 2 (Typical: 4 dBm)

Sensitivity: -80 dBm

Operating Voltage: 3.6V to 6V DC (Breakout board accepts 5V; internal chip operates at 3.3V)

Operating Current: 30 mA (Pairing), 10 mA (Connected)

Communication Range: Approximately 10 meters (33 feet)

Default Baud Rate: 9600 bps (Data Mode), 38400 bps (AT Command Mode)

Default Pairing PIN: 1234 or 0000

Default Bluetooth Name: HC-05

2. Pin Configuration

VCC: Power supply input (+5V)

GND: Ground connection

TXD: Serial data transmission pin (Connect to microcontroller RX pin)

RXD: Serial data reception pin (Connect to microcontroller TX pin through a voltage divider when using a 5V microcontroller)

STATE: Indicates connection status (LOW = Disconnected, HIGH = Connected)

EN/KEY: Used to enter AT Command Mode by setting HIGH before power-up

3. Operational Modes

- Data Mode (Default Mode)

Used for wireless serial communication between the microcontroller and a Bluetooth device. Data received through Bluetooth is transmitted directly through the serial interface and vice versa.

- AT Command Mode

Used to configure module parameters such as name, password, baud rate, and operating role. Activated by holding the EN/KEY pin HIGH before powering the module. The onboard LED blinks slowly (once every two seconds) in this mode.

4. Master and Slave Modes

- Slave Mode (Default)

Waits for incoming connection requests from smartphones, laptops, or other Bluetooth master devices. Cannot initiate connections.

- Master Mode

Searches for nearby Bluetooth devices and initiates connections automatically. Can communicate with another HC-05 or HC-06 configured as a slave.

5. Common AT Commands

Command	Function
AT	Tests communication and returns "OK"
AT+VERSION?	Displays firmware version
AT+NAME	MyDevice Changes Bluetooth device name
AT+PSWD="5678"	Changes pairing password
AT+UART=115200,0,0	Sets baud rate to 115200 bps
AT+ROLE=1	Sets module to Master Mode
AT+ROLE=0	Sets module to Slave Mode
AT+ORGL	Restores factory default settings

Note: AT commands must be sent at 38400 baud rate with both Carriage Return (CR) and Line Feed (LF) enabled in the serial monitor.

6. Applications

Wireless Arduino communication

Bluetooth-controlled robots and vehicles

Home automation systems

Wireless sensor monitoring

Industrial data acquisition

Smartphone-controlled electronic projects

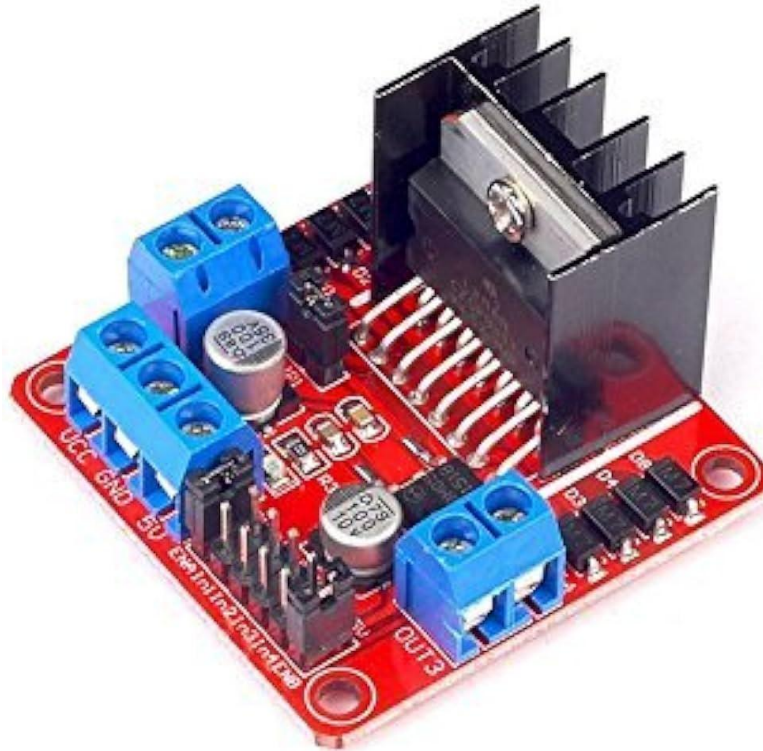
Connections:

HC-05 to Arduino Uno

- VCC -> 5V
- GND -> GND
- TXD -> Pin 2 (SoftwareSerial RX)
- RXD -> Pin 3 (SoftwareSerial TX) through a voltage divider using 1k Ohm and 2k Ohm resistors

L298N Motor Driver

A dual H-Bridge motor driver used to control motor direction and speed.



The L298N is a high-power dual H-Bridge motor driver integrated circuit widely used in embedded systems, robotics, and automation applications for controlling DC motors and stepper motors. It serves as an interface between low-power microcontrollers and high-current actuators, enabling safe and efficient motor operation. The module can drive two DC motors independently while supporting both directional control and speed regulation.

The L298N operates using an internal dual H-Bridge architecture, allowing the polarity of the voltage applied across a motor to be reversed. This enables bidirectional motor rotation, making it suitable for forward, reverse, turning, and braking operations in robotic vehicles. The driver accepts standard TTL logic signals from controllers such as the Arduino Uno, ESP32, and STM32, while handling motor supply voltages ranging from 5V to 35V and currents up to 2A per channel under suitable cooling conditions.

In this gesture-controlled robotic vehicle, the L298N functions as the primary motor control and power amplification stage. It isolates the low-current control circuitry from the high-current motor load, thereby protecting the microcontroller from electrical noise and inductive back electromotive force (Back-EMF) generated by the motors. Through its input control pins (IN1–IN4), the module manages motor direction, while the Enable pins (ENA and ENB) support Pulse Width Modulation (PWM) signals for precise speed control.

The module also incorporates protection diodes and a voltage regulator circuit, improving system reliability and simplifying power management. Due to its robustness, ease of interfacing, and ability to control multiple motors simultaneously, the L298N remains one of the most widely adopted motor drivers for educational, research, and industrial prototyping applications.

Connections : L298N to Arduino Uno

- ENA -> Pin 5 (PWM)
- IN1 -> Pin 6
- IN2 -> Pin 7
- IN3 -> Pin 8
- IN4 -> Pin 9
- ENB -> Pin 10 (PWM)

Motor Connections :

- OUT1 and OUT2 -> Left Side Motors
- OUT3 and OUT4 -> Right Side Motors

DC Geared Motors (BO Motors) :

Provide vehicle movement. Two motors are connected on the left side and two on the right side.



BO (Battery Operated) Motors are compact, lightweight, and cost-effective DC geared motors widely used in educational robotics, automation systems, and embedded electronics projects. They consist of a small permanent magnet DC motor coupled with an integrated reduction gearbox, which converts the motor's high rotational speed into lower speed and higher torque. This gear reduction mechanism enables the motor to deliver sufficient mechanical force for driving robotic vehicles and mobile platforms.

Unlike conventional DC motors that operate at very high speeds with limited torque, BO motors are specifically designed to provide the torque required for motion control applications. The internal gearbox contains multiple stages of precision gears that significantly reduce the output shaft speed while increasing the available torque. This allows robotic systems to move smoothly, overcome frictional forces, and carry the weight of the chassis, batteries, sensors, and control electronics without excessive power consumption.

In the gesture-controlled robotic vehicle, BO motors function as the primary electromechanical actuators responsible for converting electrical energy into mechanical motion. The motors receive controlled power from the L298N motor driver, which regulates both direction and speed according to commands generated by the Arduino-based control system. By varying the polarity of the applied voltage, the motors can rotate in clockwise or counterclockwise

directions, enabling forward movement, reverse movement, turning maneuvers, and pivot rotations.

The use of multiple BO motors enhances the vehicle's stability, traction, and maneuverability. Their rapid response to voltage changes allows the robotic platform to execute directional changes efficiently and accurately. Furthermore, the motors operate effectively over a wide voltage range, typically between 3V and 12V, making them highly suitable for portable battery-powered systems. Their low power consumption and reliable performance contribute to extended operating time and improved energy efficiency.

Due to their simple construction, ease of integration, low maintenance requirements, and excellent balance between speed and torque, BO motors have become one of the most commonly used actuators in robotic vehicles, line-following robots, obstacle-avoidance systems, autonomous platforms, and various educational engineering projects. Their affordability and dependable operation make them an ideal choice for developing compact mobile robotic systems.

3.2 Receiver Vehicle Hardware Interfacing Map :

VEHICLE UNIT CONNECTION MATRIX				
	SOURCE COMPONENT	PIN / TERMINAL	DESTINATION COMPONENT	PIN / TERMINAL
1	Battery (11.1V)	Positive (+)	L298N Driver	12V Screw Terminal
2	Battery (11.1V)	Negative (-)	L298N Driver	GND Screw Terminal
3	L298N Driver	GND Screw Terminal	Arduino Uno	GND Pin
4	L298N Driver	5V Output	Arduino Uno	5V Pin
5	HC-05 Module	VCC	Arduino Uno	5V Pin
6	HC-05 Module	GND	Arduino Uno	GND Pin
7	HC-05 Module	TXD	Arduino Uno	Digital Pin 2
8	Arduino Uno	Digital Pin 3	1 k Ω Resistor	Terminal A
9	1 k Ω Resistor	Terminal B	HC-05 Module	RXD Pin
10	2 k Ω Resistor	Junction Node	HC-05 Module	RXD Pin
11	2 k Ω Resistor	Ground Node	Arduino Uno	GND Pin
12	Arduino Uno	Digital Pin 5	L298N Driver	ENA Terminal
13	Arduino Uno	Digital Pin 6	L298N Driver	IN1 Terminal
14	Arduino Uno	Digital Pin 7	L298N Driver	IN2 Terminal
15	Arduino Uno	Digital Pin 8	L298N Driver	IN3 Terminal
16	Arduino Uno	Digital Pin 9	L298N Driver	IN4 Terminal
17	Arduino Uno	Digital Pin 10	L298N Driver	ENB Terminal
18	Left Motors (2x)	Positive (Spliced)	L298N Driver	OUT1 Terminal
19	Left Motors (2x)	Negative (Spliced)	L298N Driver	OUT2 Terminal
20	Right Motors (2x)	Positive (Spliced)	L298N Driver	OUT3 Terminal
	Right Motors (2x)	Negative (Spliced)	L298N Driver	OUT4 Terminal

Battery system :



Vehicle Power Infrastructure : The car utilizes three 3.7V Lithium-Ion cells wired in series (3S configuration) to produce a nominal output voltage rail of 11.1V (up to 12.6V at full charge). This higher voltage feeds the L298N driver terminals directly, maximizing the mechanical torque and speed of the BO geared motors.

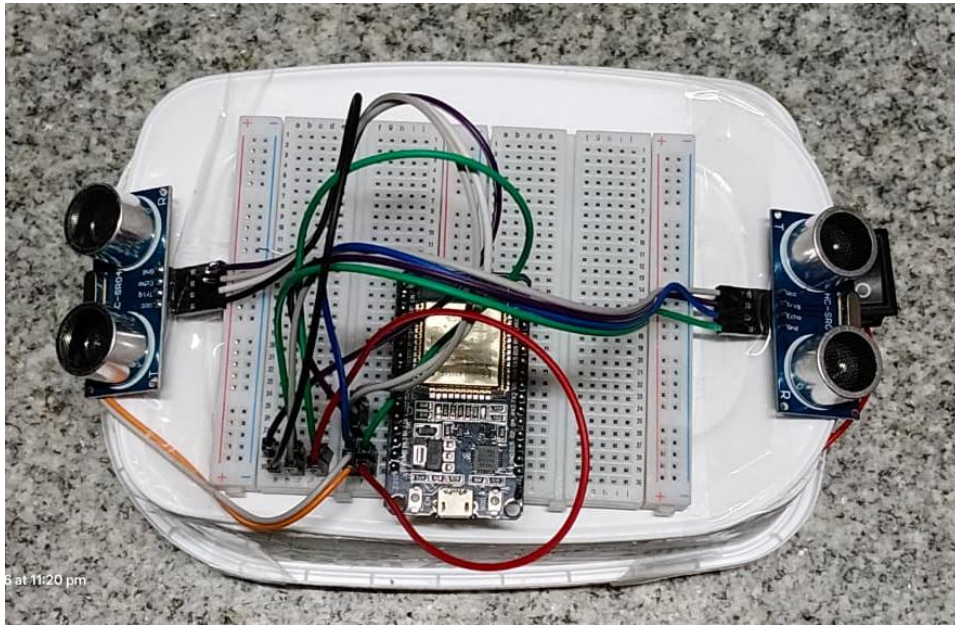
Remote Power Infrastructure : The remote controller utilizes two 3.7V Lithium-Ion cells wired in series (2S configuration) to deliver a stable nominal 7.4V (up to 8.4V at full charge). This voltage supplies the ESP32 via its VIN pin, where the onboard regulator safely steps it down to the 3.3V required for the processor and the ultrasonic sensor trigger logic.

Shared Ground Strategy : Both battery networks use a shared common ground terminal layout relative to their respective microcontrollers, preventing floating logic thresholds and ensuring that wireless data packets and PWM speed commands are parsed with absolute tracking accuracy.

4. FIRMWARE OPERATION

4.1 Transmitter Operation

1. The ESP32 continuously reads both ultrasonic sensors with a small delay between measurements to avoid signal interference.

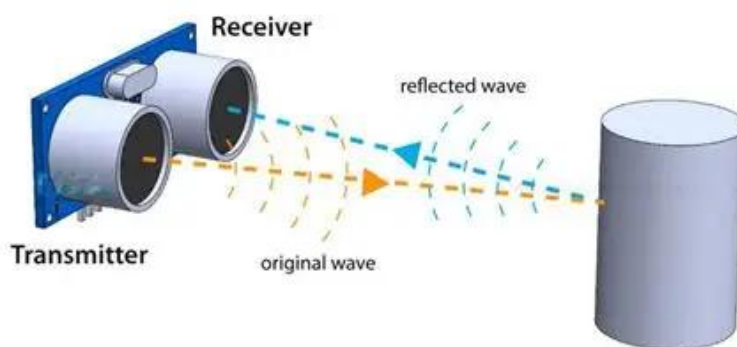


2. Distance is calculated using the speed of sound formula:

$$\text{Distance} = (\text{Echo Time} \times 0.0343) / 2$$

3. Steering Commands:

- 4 cm to 12 cm -> Right (R)
- 14 cm to 24 cm -> Left (L)
- Otherwise -> Straight (S)



4. Throttle Commands:

- 4 cm to 13 cm -> Forward movement with speed varying from 255 to 160.
- 15 m to 24 cm -> Reverse movement with speed varying from 160 to 255.
- 14 cm -> Dead Zone (Stop).

5. The ESP32 creates a data packet such as: F,210,S and transmits it wirelessly through Bluetooth.

4.2 ESP 32 CODE :

```
#include "BluetoothSerial.h"
// Check if Bluetooth is properly enabled in setup
#ifndef CONFIG_BT_ENABLED || !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth is not enabled! Please run `make menuconfig` to enable it.
#endif

BluetoothSerial SerialBT;

// Your HC-05 Receiver MAC Address
uint8_t address[6] = {0x00, 0x24, 0x08, 0x00, 0x5F, 0xED};

// --- Sensor 1 Pins (Steering) ---
const int trigPin1 = 12;
const int echoPin1 = 13;

// --- Sensor 2 Pins (Throttle) ---
const int trigPin2 = 14;
const int echoPin2 = 27;

#define SOUND_SPEED 0.0343

void setup() {
  Serial.begin(115200);

  // Start ESP32 Bluetooth Master mode
  SerialBT.begin("ESP32_Remote", true);
  Serial.println("Bluetooth Master Started. Connecting to HC-05...");

  // Connect directly using the MAC Address
  bool connected = SerialBT.connect(address);

  if (connected) {
    Serial.println("Successfully Connected to HC-05!");
  } else {
    Serial.println("Connection failed. Retrying in loop...");
    while (!SerialBT.connected()) {
      SerialBT.connect(address);
      delay(2000);
    }
  }
}
```

26

```
pinMode(trigPin1, OUTPUT);
pinMode(echoPin1, INPUT);

pinMode(trigPin2, OUTPUT);
pinMode(echoPin2, INPUT);
}

void loop() {

    // If connection drops, attempt reconnection
    if (!SerialBT.connected()) {
        Serial.println("Connection lost. Reconnecting...");
        SerialBT.connect(address);
        delay(500);
        return;
    }

    // --- Read Sensor 1 (Steering) ---
    digitalWrite(trigPin1, LOW);
    delayMicroseconds(2);

    digitalWrite(trigPin1, HIGH);
    delayMicroseconds(10);

    digitalWrite(trigPin1, LOW);

    long d1 = pulseIn(echoPin1, HIGH);
    float dist1 = d1 * SOUND_SPEED / 2;

    delay(20); // Prevent ultrasonic crosstalk

    // --- Read Sensor 2 (Throttle) ---
    digitalWrite(trigPin2, LOW);
    delayMicroseconds(2);

    digitalWrite(trigPin2, HIGH);
    delayMicroseconds(10);

    digitalWrite(trigPin2, LOW);
```

```

long d2 = pulseIn(echoPin2, HIGH);
float dist2 = d2 * SOUND_SPEED / 2;

char steering = 'S'; // Default: Straight
char direction = 'X'; // Default: Stop
int motorSpeed = 0;

// --- Evaluate Steering (Sensor 1) ---
if (dist1 >= 4.0 && dist1 <= 12.0) {
    steering = 'R';
}
else if (dist1 >= 14.0 && dist1 <= 24.0) {
    steering = 'L';
}

// --- Evaluate Throttle & Speed (Sensor 2) ---
if (dist2 < 4.0 || dist2 > 24.0 ||
    (dist2 >= 13.5 && dist2 <= 14.5)) {

    direction = 'X';
    motorSpeed = 0;
}
else if (dist2 >= 4.0 && dist2 <= 13.0) {

    direction = 'F';
    motorSpeed = map(dist2, 4, 13, 255, 120);
}
else if (dist2 >= 15.0 && dist2 <= 24.0) {

    direction = 'B';
    motorSpeed = map(dist2, 15, 24, 120, 255);
}

// --- Send Data via Bluetooth ---
// Packet string syntax: "Direction,Speed,Steering\n"

SerialBT.print(direction);
SerialBT.print(",");
SerialBT.print(motorSpeed);
SerialBT.print(",");
SerialBT.println(steering);

```

```

// Debugging to ESP32 Serial Monitor
Serial.print("Sent: ");
Serial.print(direction);
Serial.print(",");
Serial.print(motorSpeed);
Serial.print(",");
Serial.println(steering);

delay(100);
}

```

The screenshot displays the Arduino IDE 2.3.10 environment. The main editor shows a file named `miniprojectesp32.ino` with the following code:

```

81 } else if (dist1 >= 14.0 && dist1 <= 24.0) {
82   steering = 'L';
83 }
84
85 // --- Evaluate Throttle & Speed (Sensor 2) ---
86 if (dist2 < 4.0 || dist2 > 24.0 || (dist2 >= 13.5 && dist2 <= 14.5)) {
87   direction = 'X';
88   motorSpeed = 0;
89 }
90 else if (dist2 >= 4.0 && dist2 <= 13.0) {
91   direction = 'F';
92   motorSpeed = map(dist2, 4, 13, 255, 120);
93 }
94 else if (dist2 >= 15.0 && dist2 <= 24.0) {
95   direction = 'B';
96   motorSpeed = map(dist2, 15, 24, 120, 255);
97 }
98
99 // --- Send Data via Bluetooth ---
100 // Packet string syntax: "Direction,Speed,Steering\n"
101 SerialBT.print(direction);
102 SerialBT.print(",");

```

Below the editor, the `Serial Monitor` tab is active, displaying the message: "Not connected. Select a board and a port to connect automatically." The status bar at the bottom indicates the board is `DOIT ESP32 DEVKIT V1` on `COM3` and is currently `[not connected]`.

OUTPUT SAMPLE :

Bluetooth Master Started. Connecting to HC-05...

Successfully Connected to HC-05!

Sent: F,255,S

Sent: F,248,S

Sent: F,236,S

Sent: F,220,R

Sent: F,205,R

Sent: X,0,S

Sent: B,128,L

Sent: B,156,L

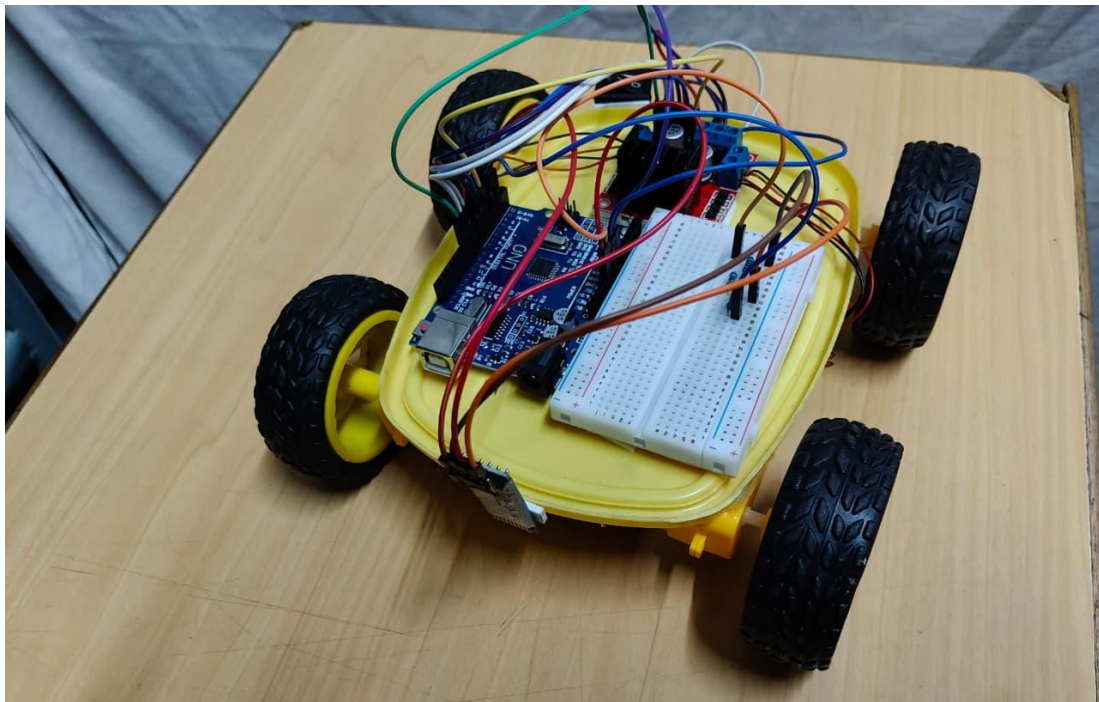
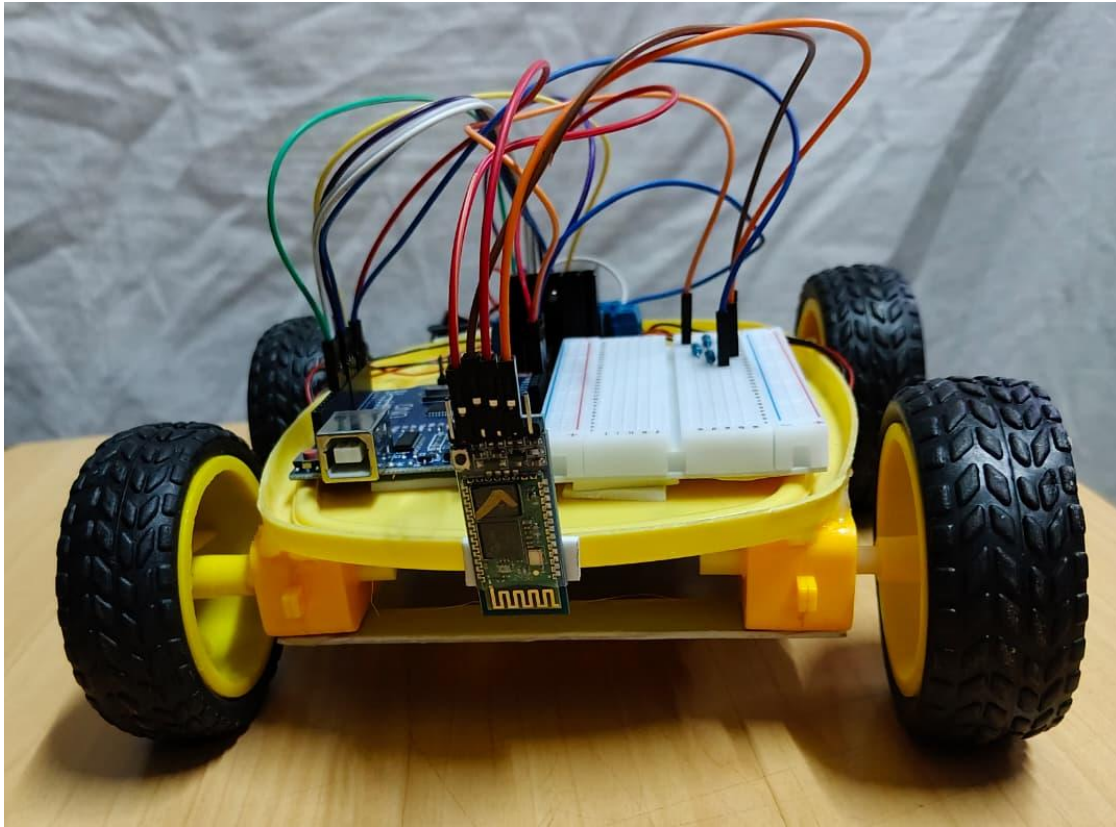
Sent: B,184,L

Sent: B,210,S

Sent: B,255,S

4.3 Receiver Operation

1. The Arduino receives Bluetooth data through the HC-05 module.



4.4 Arduino Code :

```
#include <SoftwareSerial.h>

// Software Serial for HC-05 (RX = Pin 2, TX = Pin 3)
SoftwareSerial BTSerial(2, 3);

// L298N Motor Driver Pins
const int ENA = 5; // PWM Pin Left
const int IN1 = 6;
const int IN2 = 7;
const int IN3 = 8;
const int IN4 = 9;
const int ENB = 10; // PWM Pin Right

void setup() {
  Serial.begin(115200);
  BTSerial.begin(9600); // Standard HC-05 Baud Rate
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  stopCar();
}

void loop() {
  if (BTSerial.available()) {
    // Read the packet until newline character
    String data = BTSerial.readStringUntil('\n');
    // Locate commas for data parsing
    int firstComma = data.indexOf(',');
    int secondComma = data.indexOf(',', firstComma + 1);
    if (firstComma != -1 && secondComma != -1) {
```

```

    char direction = data.charAt(0);
    int targetSpeed =
        data.substring(firstComma + 1, secondComma).toInt();
    char steering = data.charAt(secondComma + 1);
    controlMotors(direction, targetSpeed, steering);
}
}
}

```

```

void controlMotors(char direction,
    int targetSpeed,
    char steering) {
// --- PRIORITY 1: STEERING GESTURES ---
if (steering == 'R') {
    // Left Side Forward
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    // Right Side Backward
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
    int spinSpeed =
        (targetSpeed > 0) ? targetSpeed : 220;
    analogWrite(ENA, spinSpeed);
    analogWrite(ENB, spinSpeed);
}
else if (steering == 'L') {
    // Left Side Backward
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);

```

```

// Right Side Forward
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);
int spinSpeed =
    (targetSpeed > 0) ? targetSpeed : 220;
analogWrite(ENA, spinSpeed);
analogWrite(ENB, spinSpeed);
}

// --- PRIORITY 2: THROTTLE GESTURES ---

else {
    if (direction == 'F') {
        // Move Forward
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        digitalWrite(IN3, HIGH);
        digitalWrite(IN4, LOW);
        analogWrite(ENA, targetSpeed);
        analogWrite(ENB, targetSpeed);
    }
    else if (direction == 'B') {
        // Move Backward
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, HIGH);
        digitalWrite(IN3, LOW);
        digitalWrite(IN4, HIGH);
        analogWrite(ENA, targetSpeed);
        analogWrite(ENB, targetSpeed);
    }
    else {

```

```
    // Stop Car
    stopCar();
}
}
}
void stopCar() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
}
```

2. The received packet is separated into:
 - Direction
 - Speed
 - Steering
3. Steering commands receive higher priority than movement commands.
4. If Left or Right steering is detected, the vehicle performs an on-the-spot pivot turn.
5. If no steering command exists, the vehicle moves according to the received direction and speed.

4.5 SYSTEM FUNCTIONAL TABLE

Hand Position	Data Packet	Vehicle response
No Hand Detected	X,0,S	Vehicle Stops
Sensor 2 at 4 cm	F,255,S	Maximum Forward Speed
Sensor 2 at 8.5 cm	F,207,S	Medium Forward Speed
Sensor 2 at 13 cm	F,160,S	Minimum Forward Speed
Sensor 2 at 14 cm	X,0,S	Dead Zone Stop
Sensor 2 at 15 cm	B,160,S	Minimum Reverse Speed
Sensor 2 at 24 cm	B,255,S	Maximum Reverse Speed
Sensor 1 at 6 cm	X,0,R	Pivot Right Turn
Sensor 1 at 20 cm	F,205,L	Pivot Left Turn

FUTURE APPLICATIONS

1. Human–Machine Interface (HMI)

Our hands provide commands, and the robot executes them. The gesture system acts as an interface between a human and a machine.

Examples:

- Touchless control panels
- Gesture-controlled wheelchairs
- Industrial robot control systems

2. Gesture Recognition System

Our ultrasonic sensors detect hand position and convert it into commands. This is a basic form of gesture recognition, although it uses distance measurements rather than cameras or AI vision.

Examples:

- Gesture-controlled TVs
- Gesture-controlled drones
- Touchless elevators

3. Teleoperation (Teleoperated Robot)

We are remotely controlling a robot from a separate device. This is exactly what engineers call: Teleoperation or Telerobotics

Examples:

- Bomb disposal robots
- Underwater ROVs
- Space rovers
- Search-and-rescue robots

Our robot is essentially a small teleoperated ground vehicle.

4. Contactless Control System

Because no physical joystick or buttons are used, it is also a:

Contactless Human Control Interface

- This became especially important in:
- Medical equipment
- Public kiosks
- Sterile laboratory environments

5. Wireless Robotic Control System

The communication between ESP32 and HC-05 makes it a:

Wireless Robotic Vehicle Control System

CONCLUSION

The development, integration, and empirical testing of the Gesture-Controlled Teleoperated Vehicle Interface via Bluetooth were executed successfully. This project successfully demonstrates a reliable, touchless Human–Machine Interface (HMI) built on a distributed system architecture using an ESP32 master remote controller and an Arduino Uno Rev3 vehicle platform.

By utilizing ultrasonic pulse time-of-flight mathematical calculations, the remote control node successfully captures hand elevation and deflection without physical contact. The ESP32's native 2.4 GHz BluetoothSerial radio stack established a reliable, low-latency master-slave communication link directly to the vehicle's HC-05 module using a dedicated MAC address configuration (00:24:08:00:5F:ED). On the receiving side, the Arduino Uno reliably parses incoming Comma-Separated Values (CSV) data streams through SoftwareSerial communication and translates them into real-time motor control commands.

A critical achievement of this engineering architecture is the firmware's priority-driven Finite State Machine (FSM). By placing steering commands at the top of the logic cascade, the system executes an immediate zero-radius pivot turn (skid-steer maneuver), overriding any active throttle mapping when necessary. This design element ensures maximum maneuverability and rapid directional response within confined operating environments.

Furthermore, the hardware power system was optimized across both nodes. The remote control unit operates from a 7.4V (2S) Lithium-Ion battery configuration, while the vehicle platform utilizes an 11.1V (3S) series-connected Lithium-Ion battery pack. This arrangement provides sufficient voltage and current delivery to maximize the mechanical torque output of the four BO geared DC motors through the L298N dual H-bridge motor driver. Additionally, a hardware voltage-divider network was incorporated to safely reduce logic-level voltages, protecting the HC-05 communication module from potential overvoltage conditions.

Ultimately, this project demonstrates that contactless gesture-based interfaces can effectively replace conventional mechanical control devices. The successful implementation of wireless communication, ultrasonic gesture sensing, variable PWM-based speed control, and priority-based motion logic validates the feasibility of intuitive touchless robotic control systems. The engineering principles demonstrated in this prototype can be extended to advanced applications such as hazardous-environment robotics, healthcare assistive devices, industrial Automated Guided Vehicles (AGVs), remote inspection platforms, and next-generation human–robot interaction systems.